

Linux 系统编程八股文

一、常用命令

1. 进程与系统状态查看命令？

命令	功能	常用参数
ps	查看进程快照	ps aux (所有进程)、ps -ef (完整格式)
top	实时查看系统资源	按 P 按 CPU 排序, 按 M 按内存排序
htop	top 的增强版	支持鼠标、颜色、树状显示
free	查看内存使用	free -h (人类可读格式)
df	查看磁盘使用	df -h
du	查看目录/文件大小	du -sh *
netstat	查看网络连接	netstat -tlnp (TCP 监听端口)
ss	netstat 的替代	ss -tlnp (更快)
lsof	查看打开的文件	lsof -i :8080 (查看 8080 端口)
kill	发送信号给进程	kill -9 PID (强制杀死)

```
# 查看 CPU 占用前 10 的进程
ps aux --sort=-%cpu | head -10

# 查看某个进程的所有线程
ps -T -p <PID>

# 查看端口占用
netstat -tlnp | grep 8080
# 或
lsof -i :8080

# 查看进程打开的文件
lsof -p <PID>
```

2. 文本处理三剑客：grep、sed、awk？

grep：文本搜索（按行匹配）

```
# 基本搜索
grep "error" logfile.txt

# 正则搜索
grep -E "ERROR|WARN" logfile.txt
```

```
# 递归搜索目录
grep -r "TODO" ./src/

# 显示行号
grep -n "error" logfile.txt

# 反向匹配（显示不匹配的行）
grep -v "debug" logfile.txt
```

sed：流编辑器（按行处理，擅长替换、删除）

```
# 替换（只打印不修改文件）
sed 's/old/new/g' file.txt

# 替换并修改文件
sed -i 's/old/new/g' file.txt

# 删除第 3 行
sed '3d' file.txt

# 删除包含 "error" 的行
sed '/error/d' file.txt

# 只打印第 5 到 10 行
sed -n '5,10p' file.txt
```

awk：文本分析工具（按列处理，擅长格式化、计算）

```
# 打印第 1 列和第 3 列
awk '{print $1, $3}' file.txt

# 指定分隔符
awk -F: '{print $1}' /etc/passwd

# 条件过滤
awk '$3 > 100 {print $1, $3}' file.txt

# 计算总和
awk '{sum += $1} END {print sum}' file.txt

# 统计日志中每个 IP 的访问次数
awk '{print $1}' access.log | sort | uniq -c | sort -rn
```

3. 正则表达式基础？

符号	含义	示例
.	匹配任意一个字符	a.c 匹配 abc、a1c
*	前一个字符出现 0 次或多次	ab*c 匹配 ac、abc、abbc
+	前一个字符出现 1 次或多次	ab+c 匹配 abc、abbc

符号	含义	示例
<code>?</code>	前一个字符出现 0 次或 1 次	<code>ab?c</code> 匹配 <code>ac</code> 、 <code>abc</code>
<code>^</code>	行首	<code>^hello</code>
<code>\$</code>	行尾	<code>world\$</code>
<code>[]</code>	字符集合	<code>[0-9]</code> 、 <code>[a-zA-Z]</code>
<code>()</code>	分组	<code>(ab)+</code>
<code> </code>	或	<code>cat dog</code>
<code>\d</code>	数字	<code>[0-9]</code>
<code>\w</code>	单词字符	<code>[a-zA-Z0-9_]</code>
<code>\s</code>	空白字符	空格、制表符

```
# 匹配 IP 地址
grep -E "[0-9]{1,3}\.[0-9]{1,3}" file.txt

# 匹配邮箱
grep -E "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}" file.txt

# 匹配手机号
grep -E "1[3-9][0-9]{9}" file.txt
```

二、进程与线程

4. 进程和线程的区别？

对比项	进程	线程
资源拥有	独立内存空间（代码段、数据段、堆、栈）	共享进程的内存空间（除栈外）
切换开销	大（需要切换页表、TLB 刷新）	小（共享地址空间）
通信方式	IPC（管道、消息队列、共享内存）	直接读写共享内存（需同步）
健壮性	一个进程崩溃不影响其他进程	一个线程崩溃可能导致整个进程崩溃
创建方式	<code>fork()</code>	<code>pthread_create()</code>
调度单位	操作系统调度的基本单位	CPU 调度的基本单位

延伸：协程（Coroutine）

- 用户态线程，由程序自己调度，切换开销极小。
- C++20 引入原生协程支持。

5. fork() 创建进程的原理？

`fork()` 系统调用创建子进程，子进程是父进程的副本（写时复制 COW）。

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return -1;
    } else if (pid == 0) {
        // 子进程
        printf("子进程 PID: %d, 父进程 PID: %d\n", getpid(), getppid());
        return 0;
    } else {
        // 父进程
        printf("父进程 PID: %d, 子进程 PID: %d\n", getpid(), pid);
        wait(NULL); // 等待子进程结束，防止僵尸进程
    }

    return 0;
}
```

写时复制 (Copy-On-Write) :

- `fork()` 后，父子进程共享物理内存页。
- 只有当某一方尝试写入时，才复制该页，分配新的物理内存。
- 优点：减少 `fork()` 的开销，特别是 `fork()` 后立即 `exec()` 的场景。

6. 僵尸进程和孤儿进程？

僵尸进程 (Zombie) :

- 子进程已终止，但父进程未调用 `wait()` 回收。
- 进程表中仍保留条目，占用 PID 资源。
- 解决：父进程调用 `wait()` 或 `waitpid()`；父进程退出后，僵尸进程被 `init` 进程收养并回收。

孤儿进程 (Orphan) :

- 父进程先于子进程退出。
- 子进程被 `init` 进程 (PID=1) 收养。
- 无害，`init` 会自动回收。

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
```

```
pid_t pid = fork();
if (pid == 0) {
    printf("子进程运行, PID=%d\n", getpid());
    // 子进程退出, 但父进程不调用 wait, 变成僵尸进程
    return 0;
}

// 父进程休眠, 不回收子进程
sleep(10);

// 正确做法:
// waitpid(pid, NULL, 0);

return 0;
}
```

查看僵尸进程:

```
ps aux | grep Z
```

7. 进程间通信 (IPC) 有哪些方式?

方式	特点	适用场景
管道 (Pipe)	半双工, 只能用于有亲缘关系的进程	父子进程通信
命名管道 (FIFO)	半双工, 可用于任意进程	无亲缘关系进程通信
消息队列	有格式的消息, 内核维护	需要消息边界的场景
共享内存	最快, 直接映射同一块物理内存	大数据量、高频通信
信号量	用于同步, 不传输数据	配合共享内存使用
信号 (Signal)	异步通知机制	进程控制 (终止、暂停)
套接字 (Socket)	可用于不同机器间通信	网络通信、本地 IPC

8. 管道 (Pipe) 的使用?

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    if (pipe(pipefd) < 0) {
        perror("pipe");
        return -1;
    }
}
```

```

pid_t pid = fork();
if (pid == 0) {
    // 子进程: 写数据
    close(pipefd[0]); // 关闭读端
    const char *msg = "Hello from child!";
    write(pipefd[1], msg, strlen(msg));
    close(pipefd[1]);
    return 0;
}

// 父进程: 读数据
close(pipefd[1]); // 关闭写端
char buf[100];
int n = read(pipefd[0], buf, sizeof(buf));
buf[n] = '\0';
printf("父进程收到: %s\n", buf);
close(pipefd[0]);

wait(NULL);
return 0;
}

```

命名管道 (FIFO) :

```

# 创建命名管道
mkfifo mypipe

# 进程 A 写
echo "hello" > mypipe

# 进程 B 读
cat < mypipe

```

9. 共享内存的使用?

共享内存是最快的 IPC 方式，多个进程映射同一块物理内存。

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/shm.h>
#include <sys/wait.h>

#define SHM_SIZE 1024

int main() {
    // 创建共享内存
    int shmid = shmget(IPC_PRIVATE, SHM_SIZE, IPC_CREAT | 0666);
    if (shmid < 0) { perror("shmget"); return -1; }

    pid_t pid = fork();
    if (pid == 0) {
        // 子进程: 附加并写入

```

```
char *shm = (char *)shmat(shmid, NULL, 0);
strcpy(shm, "Hello from shared memory!");
shmdt(shm); // 分离
return 0;
}

// 父进程：等待子进程写入后读取
wait(NULL);
char *shm = (char *)shmat(shmid, NULL, 0);
printf("父进程读取: %s\n", shm);
shmdt(shm);
shmctl(shmid, IPC_RMID, NULL); // 删除共享内存

return 0;
}
```

注意：共享内存本身不提供同步机制，需要配合信号量或互斥锁使用。

10. 信号 (Signal) 机制？

信号是 Linux 中异步通知机制，用于进程间通信或系统事件通知。

常见信号：

信号	编号	默认动作	说明
SIGHUP	1	终止	终端断开
SIGINT	2	终止	Ctrl+C
SIGQUIT	3	核心转储	Ctrl+\
SIGKILL	9	终止	强制杀死，不可捕获
SIGSEGV	11	核心转储	段错误（非法内存访问）
SIGTERM	15	终止	优雅终止（kill 默认）
SIGCHLD	17	忽略	子进程状态改变
SIGALRM	14	终止	定时器到期

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

// 信号处理函数
void sigHandler(int sig) {
    printf("收到信号: %d\n", sig);
}

int main() {
    // 注册信号处理函数
    signal(SIGINT, sigHandler);
    signal(SIGUSR1, sigHandler);
}
```

```

printf("进程 PID: %d\n", getpid());
printf("等待信号...\n");

while (1) {
    sleep(1);
}

return 0;
}

```

sigaction (更安全的信号处理) :

```

#include <stdio.h>
#include <signal.h>

void handler(int sig, siginfo_t *info, void *context) {
    printf("收到信号 %d, 来自 PID %d\n", sig, info->si_pid);
}

int main() {
    struct sigaction sa;
    sa.sa_sigaction = handler;
    sa.sa_flags = SA_SIGINFO;
    sigemptyset(&sa.sa_mask);

    sigaction(SIGUSR1, &sa, NULL);

    printf("PID: %d\n", getpid());
    while (1) sleep(1);

    return 0;
}

```

三、多线程编程

11. POSIX 线程 (pthread) 基础?

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void *threadFunc(void *arg) {
    int id = *(int *)arg;
    printf("线程 %d 运行, 线程 ID: %lu\n", id, pthread_self());
    return NULL;
}

int main() {
    pthread_t threads[3];
    int ids[3] = {1, 2, 3};

    for (int i = 0; i < 3; i++) {

```



```

        pthread_create(&threads[i], NULL, threadFunc, &ids[i]);
    }

    for (int i = 0; i < 3; i++) {
        pthread_join(threads[i], NULL); // 等待线程结束
    }

    printf("所有线程结束\n");
    return 0;
}

```

编译: `gcc -o thread thread.c -lpthread`

12. 互斥锁 (Mutex) 的使用?

```

#include <stdio.h>
#include <pthread.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < 10000; i++) {
        pthread_mutex_lock(&mutex); // 加锁
        counter++;
        pthread_mutex_unlock(&mutex); // 解锁
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", counter); // 20000
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

死锁的四个必要条件:

1. 互斥条件
2. 持有并等待
3. 不可剥夺
4. 循环等待

避免死锁:

- 按固定顺序加锁
- 使用 `pthread_mutex_trylock()` 尝试加锁

- 设置超时

13. 条件变量 (Condition Variable) ?

条件变量用于线程间同步，通常与互斥锁配合使用。

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int ready = 0;

void *producer(void *arg) {
    sleep(1); // 模拟生产时间
    pthread_mutex_lock(&mutex);
    ready = 1;
    printf("生产者: 数据已准备好\n");
    pthread_cond_signal(&cond); // 通知消费者
    pthread_mutex_unlock(&mutex);
    return NULL;
}

void *consumer(void *arg) {
    pthread_mutex_lock(&mutex);
    while (!ready) {
        printf("消费者: 等待数据...\n");
        pthread_cond_wait(&cond, &mutex); // 等待通知 (自动释放锁)
    }
    printf("消费者: 收到数据\n");
    pthread_mutex_unlock(&mutex);
    return NULL;
}

int main() {
    pthread_t prod, cons;
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_create(&prod, NULL, producer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);
    return 0;
}
```

14. 读写锁 (Read-Write Lock) ?

读写锁允许多个读者同时读，但写者独占。

```
#include <stdio.h>
#include <pthread.h>
```

```

pthread_rwlock_t rwlock = PTHREAD_RWLOCK_INITIALIZER;
int data = 0;

void *reader(void *arg) {
    int id = *(int *)arg;
    pthread_rwlock_rdlock(&rwlock); // 读锁
    printf("读者 %d 读取数据: %d\n", id, data);
    sleep(1);
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}

void *writer(void *arg) {
    int id = *(int *)arg;
    pthread_rwlock_wrlock(&rwlock); // 写锁
    data += 10;
    printf("写者 %d 写入数据: %d\n", id, data);
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}

int main() {
    pthread_t readers[3], writers[2];
    int ids[] = {1, 2, 3};

    for (int i = 0; i < 3; i++)
        pthread_create(&readers[i], NULL, reader, &ids[i]);
    for (int i = 0; i < 2; i++)
        pthread_create(&writers[i], NULL, writer, &ids[i]);

    for (int i = 0; i < 5; i++)
        pthread_join(readers[i < 3 ? i : i - 3], NULL);

    pthread_rwlock_destroy(&rwlock);
    return 0;
}

```

15. 线程池的实现?

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <queue>

typedef struct {
    void (*func)(void *arg);
    void *arg;
} Task;

class ThreadPool {
    pthread_t *threads;
    std::queue<Task> taskQueue;
    pthread_mutex_t mutex;
    pthread_cond_t cond;

```

```

bool stop;
int numThreads;

static void *workerFunc(void *arg) {
    ThreadPool *pool = (ThreadPool *)arg;
    while (true) {
        Task task;
        pthread_mutex_lock(&pool->mutex);
        while (pool->taskQueue.empty() && !pool->stop) {
            pthread_cond_wait(&pool->cond, &pool->mutex);
        }
        if (pool->stop && pool->taskQueue.empty()) {
            pthread_mutex_unlock(&pool->mutex);
            return NULL;
        }
        task = pool->taskQueue.front();
        pool->taskQueue.pop();
        pthread_mutex_unlock(&pool->mutex);
        task.func(task.arg);
    }
}

public:
    ThreadPool(int n) : numThreads(n), stop(false) {
        threads = new pthread_t[n];
        pthread_mutex_init(&mutex, NULL);
        pthread_cond_init(&cond, NULL);
        for (int i = 0; i < n; i++)
            pthread_create(&threads[i], NULL, workerFunc, this);
    }

    void addTask(void (*func)(void *), void *arg) {
        Task task = {func, arg};
        pthread_mutex_lock(&mutex);
        taskQueue.push(task);
        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);
    }

    ~ThreadPool() {
        pthread_mutex_lock(&mutex);
        stop = true;
        pthread_cond_broadcast(&cond);
        pthread_mutex_unlock(&mutex);
        for (int i = 0; i < numThreads; i++)
            pthread_join(threads[i], NULL);
        delete[] threads;
        pthread_mutex_destroy(&mutex);
        pthread_cond_destroy(&cond);
    }
};

void taskFunc(void *arg) {
    int id = *(int *)arg;
    printf("任务 %d 在运行\n", id);
    sleep(1);
}

```

```

}

int main() {
    ThreadPool pool(4);
    int ids[10];
    for (int i = 0; i < 10; i++) {
        ids[i] = i;
        pool.addTask(taskFunc, &ids[i]);
    }
    sleep(3);
    return 0;
}

```

四、I/O 多路复用

16. select、poll、epoll 的区别？

对比项	select	poll	epoll
最大文件描述符	1024 (FD_SETSIZE)	无限制	无限制
数据结构	fd_set 位图	pollfd 数组	红黑树 + 链表
时间复杂度	O(n)	O(n)	O(1) (事件驱动)
工作方式	水平触发 (LT)	水平触发 (LT)	LT + 边缘触发 (ET)
内核到用户空间	每次复制整个 fd_set	每次复制整个数组	只返回就绪的 fd
适用场景	连接数少、跨平台	连接数中等	高并发、Linux 专用

17. select 的使用？

```

#include <stdio.h>
#include <sys/select.h>
#include <unistd.h>

int main() {
    fd_set readfds;
    int maxfd = 0;

    FD_ZERO(&readfds);
    FD_SET(STDIN_FILENO, &readfds);
    maxfd = STDIN_FILENO;

    printf("等待输入...\n");

    struct timeval timeout;
    timeout.tv_sec = 5;
    timeout.tv_usec = 0;

    int ret = select(maxfd + 1, &readfds, NULL, NULL, &timeout);
    if (ret > 0 && FD_ISSET(STDIN_FILENO, &readfds)) {

```

```

    char buf[100];
    read(STDIN_FILENO, buf, sizeof(buf));
    printf("收到: %s", buf);
} else if (ret == 0) {
    printf("超时\n");
}

return 0;
}

```

18. epoll 的使用?

```

#include <stdio.h>
#include <sys/epoll.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int epollfd = epoll_create1(0);
    if (epollfd < 0) { perror("epoll_create"); return -1; }

    struct epoll_event ev, events[10];
    ev.events = EPOLLIN;
    ev.data.fd = STDIN_FILENO;
    epoll_ctl(epollfd, EPOLL_CTL_ADD, STDIN_FILENO, &ev);

    printf("等待输入 (epoll) ...\n");

    int nfds = epoll_wait(epollfd, events, 10, 5000);
    for (int i = 0; i < nfds; i++) {
        if (events[i].data.fd == STDIN_FILENO) {
            char buf[100];
            read(STDIN_FILENO, buf, sizeof(buf));
            printf("收到: %s", buf);
        }
    }

    close(epollfd);
    return 0;
}

```

epoll 的两种触发模式:

- **LT (Level Triggered, 水平触发)** : 默认模式, 只要 fd 可读/可写, 就会持续通知。
- **ET (Edge Triggered, 边缘触发)** : 只在状态变化时通知一次, 必须一次性读完/写完所有数据。

```

// ET 模式示例
ev.events = EPOLLIN | EPOLLET; // 边缘触发
epoll_ctl(epollfd, EPOLL_CTL_ADD, fd, &ev);

```

五、生产者消费者模型

19. 生产者消费者模型的完整实现？

```
#include <stdio.h>
#include <pthread.h>
#include <queue>
#include <unistd.h>

#define BUFFER_SIZE 10

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t notFull = PTHREAD_COND_INITIALIZER;
pthread_cond_t notEmpty = PTHREAD_COND_INITIALIZER;
std::queue<int> buffer;

void *producer(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < 5; i++) {
        pthread_mutex_lock(&mutex);
        while (buffer.size() >= BUFFER_SIZE) {
            printf("生产者 %d: 缓冲区满, 等待...\n", id);
            pthread_cond_wait(&notFull, &mutex);
        }
        buffer.push(i);
        printf("生产者 %d 生产: %d\n", id, i);
        pthread_cond_signal(&notEmpty);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
    return NULL;
}

void *consumer(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < 5; i++) {
        pthread_mutex_lock(&mutex);
        while (buffer.empty()) {
            printf("消费者 %d: 缓冲区空, 等待...\n", id);
            pthread_cond_wait(&notEmpty, &mutex);
        }
        int val = buffer.front();
        buffer.pop();
        printf("消费者 %d 消费: %d\n", id, val);
        pthread_cond_signal(&notFull);
        pthread_mutex_unlock(&mutex);
        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;
    int pid = 1, cid = 1;
    pthread_create(&prod, NULL, producer, &pid);
```

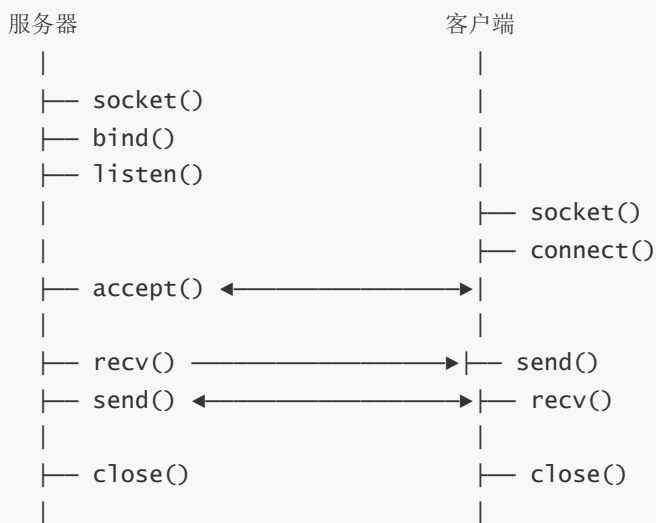
```

pthread_create(&cons, NULL, consumer, &cid);
pthread_join(prod, NULL);
pthread_join(cons, NULL);
pthread_mutex_destroy(&mutex);
pthread_cond_destroy(&notFull);
pthread_cond_destroy(&notEmpty);
return 0;
}

```

六、网络服务器编程

20. TCP 服务器编程流程？



```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    // 1. 创建 socket
    int serverFd = socket(AF_INET, SOCK_STREAM, 0);
    if (serverFd < 0) { perror("socket"); return -1; }

    // 2. 设置地址复用
    int opt = 1;
    setsockopt(serverFd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    // 3. 绑定地址
    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(PORT);
}

```



```

    if (bind(serverFd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind"); return -1;
    }

    // 4. 监听
    if (listen(serverFd, 10) < 0) { perror("listen"); return -1; }
    printf("服务器启动, 监听端口 %d\n", PORT);

    // 5. 接受连接
    struct sockaddr_in clientAddr;
    socklen_t clientLen = sizeof(clientAddr);
    int clientFd = accept(serverFd, (struct sockaddr *)&clientAddr, &clientLen);
    if (clientFd < 0) { perror("accept"); return -1; }
    printf("客户端连接: %s:%d\n", inet_ntoa(clientAddr.sin_addr),
        ntohs(clientAddr.sin_port));

    // 6. 收发数据
    char buffer[BUFFER_SIZE];
    while (1) {
        int n = read(clientFd, buffer, sizeof(buffer));
        if (n <= 0) break;
        buffer[n] = '\0';
        printf("收到: %s", buffer);
        write(clientFd, buffer, n); // 回显
    }

    // 7. 关闭
    close(clientFd);
    close(serverFd);
    return 0;
}

```

21. epoll 实现高并发服务器?

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <fcntl.h>

#define PORT 8080
#define MAX_EVENTS 64
#define BUFFER_SIZE 1024

void setNonBlocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

int main() {

```

```

int serverFd = socket(AF_INET, SOCK_STREAM, 0);
int opt = 1;
setsockopt(serverFd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

struct sockaddr_in addr = {
    .sin_family = AF_INET,
    .sin_addr.s_addr = INADDR_ANY,
    .sin_port = htons(PORT)
};
bind(serverFd, (struct sockaddr *)&addr, sizeof(addr));
listen(serverFd, 10);
setNonBlocking(serverFd);

int epollFd = epoll_create1(0);
struct epoll_event ev, events[MAX_EVENTS];
ev.events = EPOLLIN;
ev.data.fd = serverFd;
epoll_ctl(epollFd, EPOLL_CTL_ADD, serverFd, &ev);

printf("epoll 服务器启动, 端口 %d\n", PORT);

while (1) {
    int nfds = epoll_wait(epollFd, events, MAX_EVENTS, -1);
    for (int i = 0; i < nfds; i++) {
        if (events[i].data.fd == serverFd) {
            // 新连接
            struct sockaddr_in clientAddr;
            socklen_t len = sizeof(clientAddr);
            int clientFd = accept(serverFd, (struct sockaddr *)&clientAddr,
&len);

            setNonBlocking(clientFd);
            ev.events = EPOLLIN | EPOLLET;
            ev.data.fd = clientFd;
            epoll_ctl(epollFd, EPOLL_CTL_ADD, clientFd, &ev);
            printf("新连接: %s:%d\n", inet_ntoa(clientAddr.sin_addr),
ntohs(clientAddr.sin_port));
        } else {
            // 数据到达
            int fd = events[i].data.fd;
            char buffer[BUFFER_SIZE];
            int n;
            while ((n = read(fd, buffer, sizeof(buffer))) > 0) {
                write(fd, buffer, n); // 回显
            }
            if (n == 0) {
                printf("客户端断开\n");
                epoll_ctl(epollFd, EPOLL_CTL_DEL, fd, NULL);
                close(fd);
            }
        }
    }
}

close(epollFd);
close(serverFd);
return 0;

```

```
}
```

22. Reactor 模型?

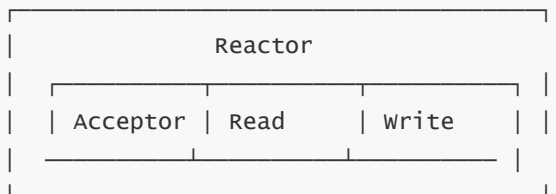
Reactor 是事件驱动的网络编程模型，核心组件：

- **Reactor**：事件分发器，监听 I/O 事件
- **Handler**：事件处理器，处理具体的读写操作
- **Acceptor**：接受新连接

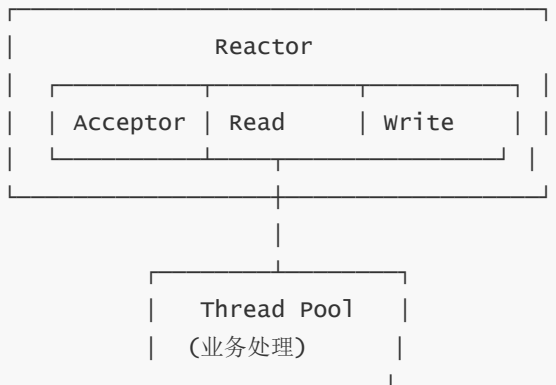
三种 Reactor 模型：

1. **单 Reactor 单线程**：所有操作在一个线程完成（简单，性能低）
2. **单 Reactor 多线程**：Reactor 在一个线程，业务处理在多线程池（常用）
3. **多 Reactor 多线程**：MainReactor 处理连接，SubReactor 处理 I/O，业务处理在多线程池（高性能，Netty 采用）

单 Reactor 单线程：



单 Reactor 多线程：



七、综合面试题

23. 进程和线程的调度区别？

- **进程调度**：操作系统内核负责，涉及上下文切换（保存/恢复寄存器、页表切换、TLB 刷新）。
- **线程调度**：
 - 内核级线程：由操作系统调度，切换开销小于进程。
 - 用户级线程（协程）：由用户程序调度，切换开销极小。

上下文切换开销：

- 进程切换：约 1000-10000 纳秒

- 线程切换：约 100-1000 纳秒
- 协程切换：约 10-100 纳秒

24. 如何查看和修改文件权限？

```
# 查看权限
ls -l file.txt
# -rw-r--r-- 1 user group 1024 Jan 1 12:00 file.txt
# 权限位：所有者(u) 组(g) 其他(o)
# r=4, w=2, x=1

# 数字方式修改
chmod 755 file.txt    # rwxr-xr-x
chmod 644 file.txt    # rw-r--r--

# 符号方式修改
chmod u+x file.txt    # 给所有者加执行权限
chmod g-w file.txt    # 给组去掉写权限
chmod o=r file.txt    # 设置其他用户只读

# 递归修改目录
chmod -R 755 /path/to/dir

# 修改所有者
chown user:group file.txt
```

25. 硬链接和软链接的区别？

对比项	硬链接	软链接（符号链接）
本质	指向同一个 inode	指向文件路径
跨文件系统	不可以	可以
目录链接	不可以	可以
源文件删除	仍然有效	失效（悬空链接）
inode	相同	不同

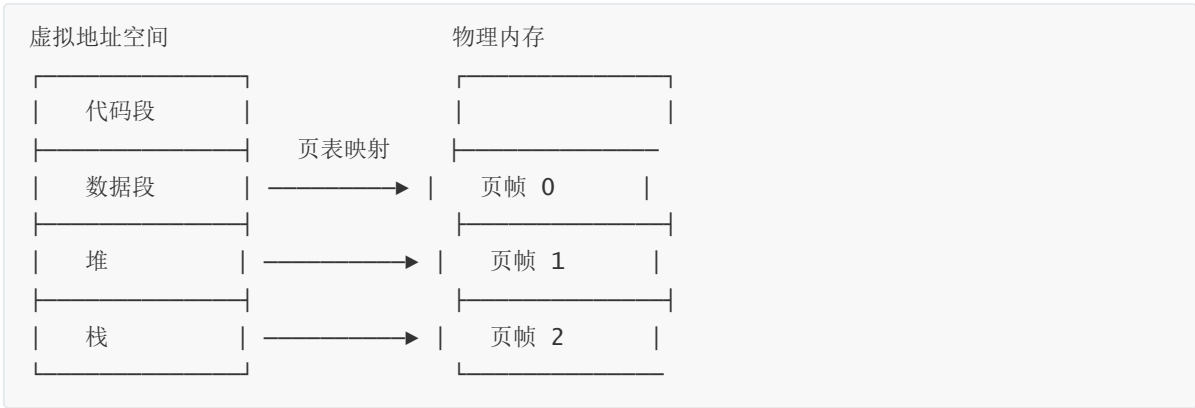
```
# 创建硬链接
ln file.txt hardlink.txt

# 创建软链接
ln -s file.txt softlink.txt

# 查看 inode
ls -li file.txt hardlink.txt
```

26. 虚拟内存和物理内存的关系？

- **虚拟内存**：每个进程有独立的虚拟地址空间（32 位系统 4GB，64 位系统更大）。
- **物理内存**：实际的 RAM 硬件。
- **页表**：将虚拟地址映射到物理地址。
- **分页**：虚拟内存和物理内存都分成固定大小的页（通常 4KB）。
- **缺页中断**：访问的页不在物理内存中，触发中断，从磁盘加载。



27. 如何排查 Linux 性能问题？

CPU 问题：

```
top          # 查看 CPU 占用
mpstat       # 多核 CPU 统计
perf top     # 性能分析
```

内存问题：

```
free -h      # 内存使用
vmstat       # 虚拟内存统计
dmesg | grep -i oom # 查看 OOM killer
```

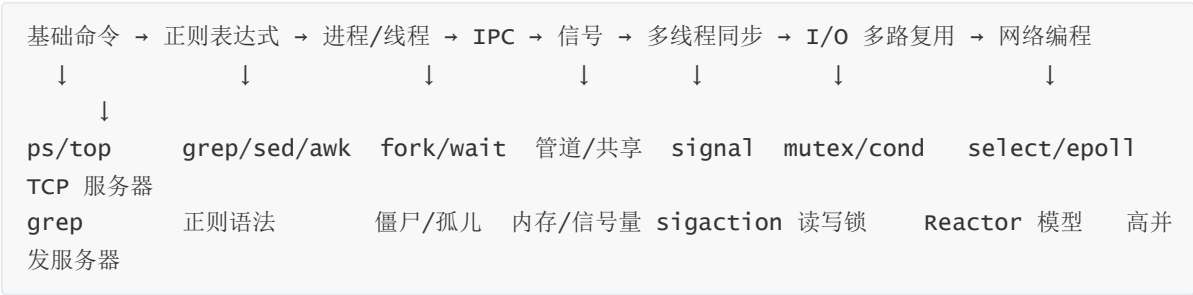
磁盘 I/O 问题：

```
iostat       # 磁盘 I/O 统计
iotop        # 进程级 I/O 监控
```

网络问题：

```
netstat -s   # 网络统计
tcpdump      # 抓包
ping/traceroute # 网络连通性
```

附录：Linux 系统编程学习路线图



面试高频问题速查

问题	核心答案
进程 vs 线程	进程独立内存，线程共享内存
fork() 原理	写时复制，子进程是父进程副本
僵尸进程	子进程退出父进程未 wait
IPC 方式	管道、消息队列、共享内存、信号量、信号
互斥锁 vs 读写锁	互斥锁独占，读写锁多读单写
select vs epoll	select O(n) 有上限，epoll O(1) 事件驱动
LT vs ET	LT 持续通知，ET 只通知一次
Reactor 模型	事件驱动，单/多 Reactor + 线程池